Note: ALL code must match the exact specification listed below (name, type of information taken as a parameter, type of information returned). All code must be your own work (use of generative AI of any form, including line code completion, is not considered your own work), and you may be examined in class before marks are awarded. Failure to demonstrate understanding during this will result in a 0 for the submission.

All code is expected to be rigourous and well-designed, **with appropriate validation present throughout**. Reminder: You may **not** use any built-in classes to carry out this logic (other than System.arraycopy()). This includes, but is not limited to, the ArrayList & Arrays classes. Use of built-in components will result in 0 for those sections.

List Specifications:

You are required to produce a array-based implementation of a List to store **Strings.** Your list should have a **standard initial capacity of 10 elements, and an expansion factor of 2.** It should provide implementations of the following:

1. **size**() -> takes no parameters & returns the number of elements currently stored in your list.
2. **get**() -> takes a single parameter, an int representing the index at which data should be retrieved. The method should return the data in the index specified.
3. **indexOf**() -> takes one parameter, an element to be found in the list. The method should return an int indicating the position at which the supplied object could be found. If the object could not be found, an appropriate value should be returned to indicate this.
    a. *Note: You can use String's equals() method to check if two Strings are the same.*
4. **add**() -> takes a single parameter, an element to be added. This element should be added to the end of the list. Null values should not be allowed. The method should return nothing.
5. **add**() -> takes in **two** parameters: an element to be added and the index at which it should be inserted. Null values should not be allowed. The method should add the value at the specified index, shifting all data appearing after it towards the end of the list. The method should return nothing, and no data should be lost.
6. **set()** -> takes two parameters: an element to be added and an index at which the element should be added. Null values should not be allowed. The method should replace the element at the specified index with the supplied element and return the element that was replaced, i.e. the data that previously occupied the specified index.
7. **remove**() -> takes a single parameter: the index from which to remove. The method should remove the element at the specified index and return it.
8. **removeAll**() -> takes a single parameter: an element to be removed. The method should remove all instances of the specified element and should return an int indicating how many instances of that element were removed.
9. **clear() ->** takes no parameters and resets the list to its original settings. The method should return nothing.
10. **deduplicate()** -> takes no parameters and generates a new list containing a single instance of each element in the current list. The original list should not be edited, & the method should return the filtered version of the list.
11. **join()** -> takes a single parameter: a list of the same type as your class, i.e. a DynamicArray object where the method is in a DynamicArray class. The method should add all elements from the parameter list to the current list and should return nothing.

Note: Validation of parameters should be included in all methods where appropriate.

Documentation:

You must include a Javadoc comment for every method (and constructor) you write. These comments should employ the standard Javadoc tags and contain all appropriate information explaining the functionality parameters, return data and any exceptions thrown (along with why they might be thrown). You are **not required to comment every line of code** you are producing. Instead, you should use in-line comments only on **complex steps** in the algorithms you're implementing.

Testing:

You have been provided with a set of Junit tests for the methods listed in this assessment. You should run these tests as you complete your code to confirm that the logic functions correctly. I've split the tests into separate files based on linked methods (i.e. the tests use certain methods to function correctly). The comment at the top of each class indicates which methods must be completed before you can run the tests from that class.

Deliverables:

A URL for your ongoing CA git repo (you should make this a private repo and add me as a collaborator). You are free to work within your main git repo for Data Structures, but if so, create a dedicated package for CA in which to work.

Deadline: October 19th, 23:59